

Healthcare Fraud Detection

Final Project Report

Team 3 — Rahim Manji

BAN 674: Data Mining | Spring 2026

Executive Summary

Healthcare fraud quietly costs insurers and policyholders billions every year. When fraudulent claims slip through, insurers absorb the loss and pass it back to everyone through higher premiums and slower claim reviews. Our project set out to see how well a few common data mining methods could spot fraudulent claims in a realistic claims dataset, and what that would mean for an insurance company that wanted to put one of these models to work.

We worked with a dataset of 10,000 healthcare insurance claims, of which about 8.3% were flagged as fraudulent. We tried three approaches: logistic regression, a decision tree, and k-nearest neighbors. After cleaning the data, addressing the class imbalance, and putting all three through the same evaluation, logistic regression came out as the clear winner. It caught roughly 96% of fraudulent claims in our test set with about 81% precision, meaning that when it flags a claim as fraud, it's right about 4 out of 5 times. For an insurer, that's a strong starting point for a screening tool that an investigator could use to prioritize which claims to look at first.

Beyond raw performance, we also looked at what the model was actually learning. The biggest signals were the relationship between the amount billed and the amount approved, the gap between when the service happened and when the claim was filed, and certain combinations of provider specialty and diagnosis. These are the kinds of things a human investigator would also look at, which gave us some confidence that the model wasn't just memorizing noise.

1. Introduction and Business Problem

Healthcare fraud is one of those problems that affects almost everyone with insurance, even if most people never think about it. The FBI estimates that healthcare fraud costs the U.S. tens of billions of dollars each year, and those costs don't disappear; they get folded back into premiums, deductibles, and the general friction of dealing with an insurance company. There's also a second cost that gets less attention: as insurers ramp up scrutiny to protect themselves from fraud, legitimate patients end up waiting longer for claims to clear or getting flagged for unnecessary reviews.

For insurers, this puts them in a tricky spot. Letting too much fraud through erodes margins and forces premium increases. Reviewing every single claim manually isn't realistic given the volume. The middle ground is to build a screening system that can quickly flag the small fraction of claims most likely to be fraudulent, so investigators can focus their attention there. That's exactly the kind of problem data mining is well-suited for.

For this project, our goal was to build and compare predictive models that could classify claims as fraudulent or not, and then think through what the results would actually mean for someone running a claims department. We deliberately kept the dataset modest in size, since the syllabus encouraged us to focus on careful analysis rather than scale.

2. The Dataset

We used the Healthcare Fraud Detection dataset from Kaggle, uploaded by user nudratabbas. It is freely available at <https://www.kaggle.com/datasets/nudratabbas/healthcare-fraud-detection-dataset> and downloads as a single CSV with no special access required.

The dataset contains 10,000 claims and 20 columns. Each row is one insurance claim. The columns mix together patient demographics (age, gender, state), clinical details (diagnosis code, procedure code, visit type, length of stay, chronic condition flag, prior visits in the past year), financial details (claim amount, approved amount), and provider information (provider ID, specialty, number of claims that provider submitted in the same month). The target variable is `Is_Fraud`, which is binary: 1 means fraudulent, 0 means legitimate.

Out of 10,000 claims, only 829 are labeled as fraudulent, which works out to about 8.3% of the data. This kind of imbalance is normal for fraud problems; by design, fraud is supposed to be rare. But it does affect how we build and evaluate models, which we'll come back to in a moment. We also noticed that three columns (`Insurance_Type`, `Provider_Specialty`, and `Prior_Visits_12m`) each had 350 missing values, which we handled during preprocessing.

2.1 Variables We Dropped

Before any modeling, we removed four columns. `Provider_ID` and `Claim_ID` are just unique identifiers with no real predictive value. `Claim_Submission_Date` was a calendar date we chose not to feature-engineer for this project.

The fourth one is more interesting. `Claim_Status` takes three values (Approved, Rejected, and Pending) and at first it looks like a useful predictor. But when we looked at the fraud rate inside each category, we found that Approved claims had only a 2.2% fraud rate while Pending and Rejected claims sat around 23% each. That's a huge gap, and it almost certainly comes from the fact that `Claim_Status` is decided as part of the same review process that produces the fraud label, which means the model would basically be cheating by reading the answer. We flagged this as a leakage risk in our interim report and decided to drop the variable so our results reflect what the model could actually do at the point a claim is first submitted, before any review has happened.

3. Exploring the Data

Before building any models, we spent some time looking at how the variables relate to fraud. A few things stood out.

Claim amount is the most obvious signal. Fraudulent claims tend to be billed at much higher amounts than legitimate ones, and the gap between claim amount and approved amount is wider for fraudulent claims. Provider behavior also matters. Providers who submit a high number of claims in a single month

are more likely to have fraud in their mix, which lines up with what investigators in real-world fraud cases tend to look for.

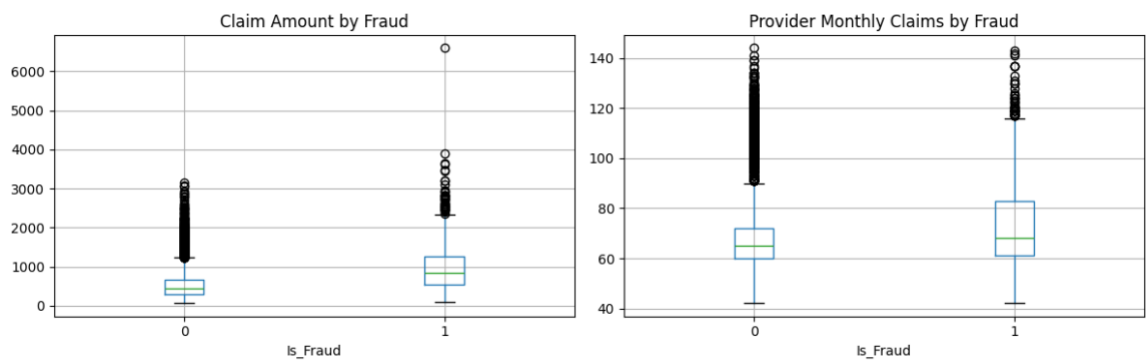


Figure 1. Claim amount and provider monthly claim volume, split by fraud label.

The correlation between the numeric variables was generally low, with one notable exception: claim amount and approved amount were strongly correlated, which makes sense (the approved amount is a function of what was billed). That suggested those two variables together, rather than separately, would carry most of the financial signal.

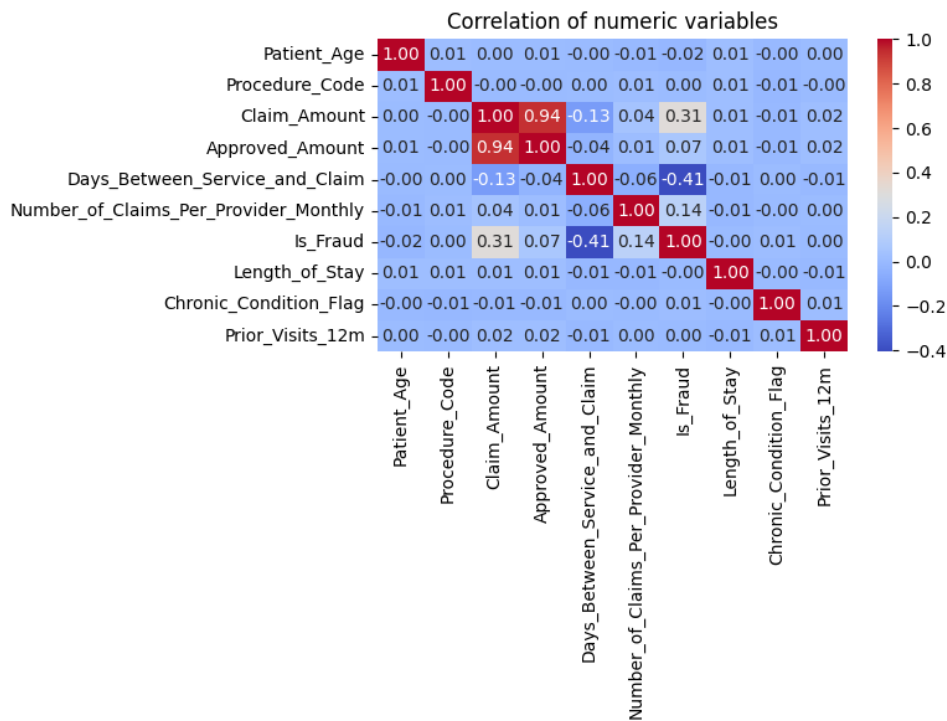


Figure 2. Correlation heatmap of numeric variables.

4. Methodology

4.1 Preparing the Data

After dropping the four columns mentioned earlier, we split the data into training (70%) and test (30%) sets, using stratified sampling so the fraud rate stayed the same in both. We then put together a

preprocessing pipeline that imputed missing values (median for numeric variables, most-frequent for categorical), scaled the numeric features to a common range (important for kNN, which is sensitive to scale), and one-hot encoded the categorical variables.

4.2 Handling Class Imbalance

Since fraud is only 8.3% of the data, a model that just predicted "not fraud" every single time would still be right 91.7% of the time. That's a useless model in practice, but it would look great if you only judged it on accuracy. To get the models to actually pay attention to the fraud class, we used random oversampling on the training data only. This duplicates fraud examples until the training set is balanced 50/50, while leaving the test set untouched so our evaluation still reflects real-world conditions. We chose this approach over SMOTE since it's the same technique we used in our k-Nearest Neighbors homework and it played well with the categorical variables in our dataset.

4.3 Models

We compared three models, each representing a different way of thinking about the classification problem:

Logistic regression was our baseline. It's interpretable, it gives us coefficients we can read directly, and it's a natural starting point for binary classification problems. Decision tree was our second model. Trees can capture non-linear interactions between variables that logistic regression would miss, and they have the added benefit of being explainable; an investigator can follow the splits to see why a claim got flagged. We capped the depth at 5 to keep the tree from overfitting. k-Nearest Neighbors was our third model. It's non-parametric, so it doesn't assume any particular shape for the underlying relationship, and it's a good test of whether the fraud cases cluster together in the feature space. We tried several values of k and picked the one that gave the best F1 score on our test set.

4.4 How We Evaluated Them

Because of the imbalance, accuracy by itself isn't a useful measure. We focused on four metrics. Precision tells us, out of all the claims a model flags as fraud, what percentage really are. Recall tells us, out of all the actual fraud cases, what percentage the model catches. F1 score is a balance of the two. And ROC AUC measures how well the model can rank claims from least to most likely to be fraud, independent of any particular cutoff.

In a fraud context, recall is usually the metric that matters most. Missing a fraudulent claim costs the insurer the full claim amount. Flagging a legitimate one just means an investigator spends a few minutes reviewing it. So our preference, all else equal, was for models that caught more fraud, even at the cost of some extra false alarms.

5. Results

5.1 Logistic Regression

Logistic regression with oversampling caught 238 of the 249 fraud cases in our test set, missing only 11. Precision was 81%, meaning that out of every 100 claims it flagged, about 81 really were fraud and 19 were false alarms. Overall F1 was 0.88 and ROC AUC was 0.998, which is a very strong ranking ability.

Table 1. Logistic Regression confusion matrix on the test set (3,000 claims).

	Predicted: Not Fraud	Predicted: Fraud
Actual: Not Fraud	2,695	56
Actual: Fraud	11	238

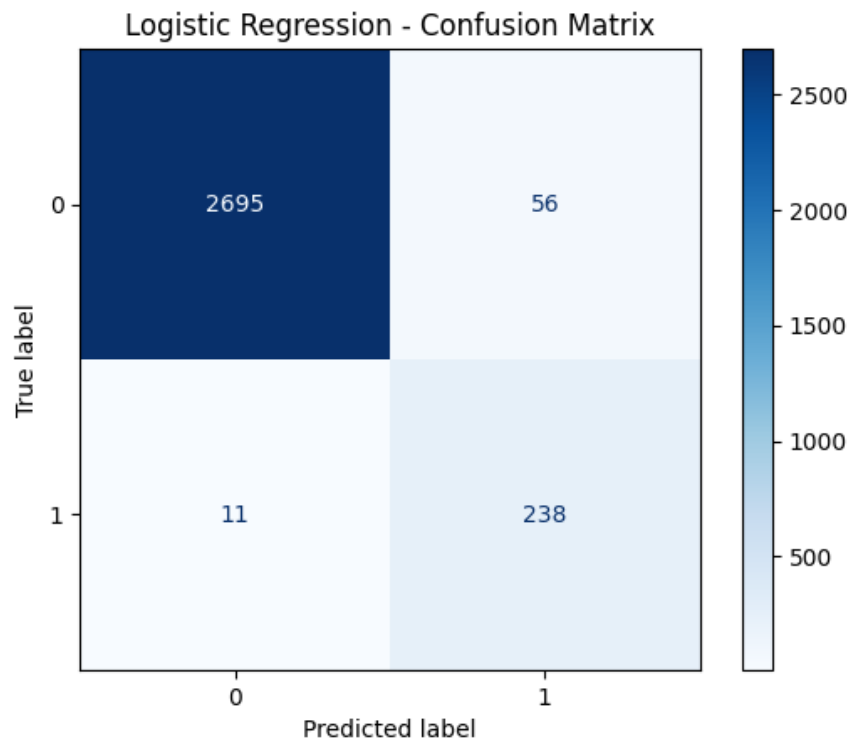


Figure 3. Logistic Regression confusion matrix.

5.2 Decision Tree

The decision tree caught about the same number of fraud cases (239 out of 249), but it did so at the cost of a lot more false positives, meaning 266 legitimate claims got flagged as fraud. Precision dropped to 47%, meaning more than half of what the tree flagged was wrong. That's a real operational problem. If investigators are wading through false alarms more than half the time, the system loses credibility and people start ignoring its flags.

On the upside, the tree is much easier to explain than logistic regression. You can literally walk down the branches and read out the rules the model is using, which has real value in a fraud investigation context where decisions sometimes have to be defended in writing.

Table 2. Decision Tree confusion matrix.

	Predicted: Not Fraud	Predicted: Fraud
Actual: Not Fraud	2,485	266
Actual: Fraud	10	239

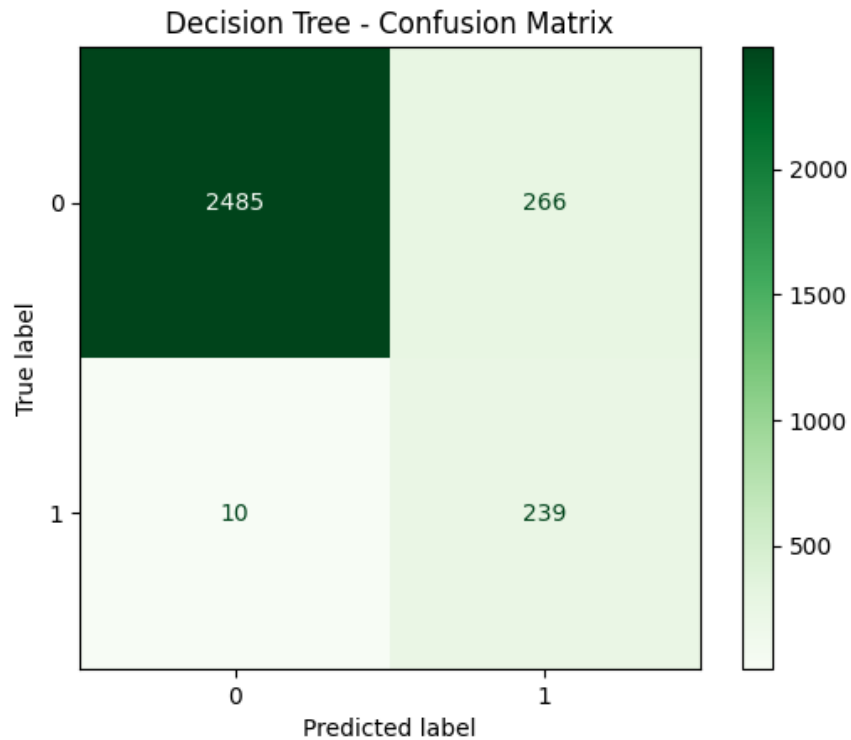


Figure 4. Decision Tree confusion matrix.

5.3 k-Nearest Neighbors

kNN was the weakest of the three. We tried k values from 3 through 21. Smaller values of k gave better F1 but with poor recall, while larger values picked up more fraud but had even worse precision. We settled on k=3 since it gave the best F1 of 0.45, but even at its best, kNN flagged 216 false positives and missed 116 actual fraud cases out of 249.

Our read on why kNN underperformed: fraud patterns in this dataset look like they're driven by specific combinations of features (high claim amount AND certain provider behaviors AND specific visit types) rather than by overall "closeness" in feature space. Distance-based methods like kNN have a harder time picking up on these conditional patterns than logistic regression or trees do.

Table 3. kNN (k=3) confusion matrix.

	Predicted: Not Fraud	Predicted: Fraud
Actual: Not Fraud	2,535	216
Actual: Fraud	116	133

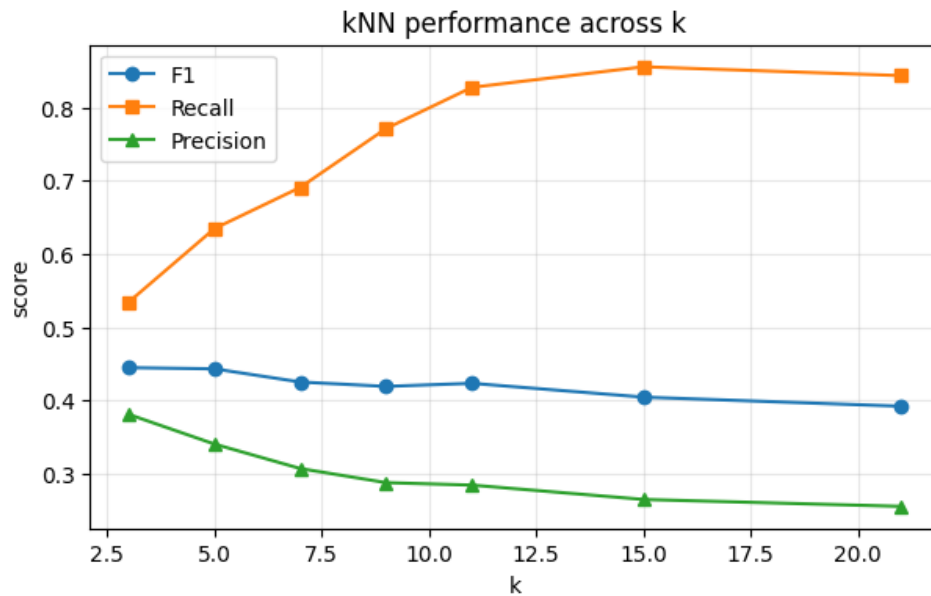


Figure 5. kNN performance across different values of k.

6. Side-by-Side Comparison

Putting the three models on the same scoreboard makes it easy to see which one to pick.

Table 4. Model performance summary on the test set.

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Logistic Regression	97.8%	81.0%	95.6%	0.877	0.998
Decision Tree	90.8%	47.3%	96.0%	0.634	0.973
kNN (k=3)	88.9%	38.1%	53.4%	0.445	0.769

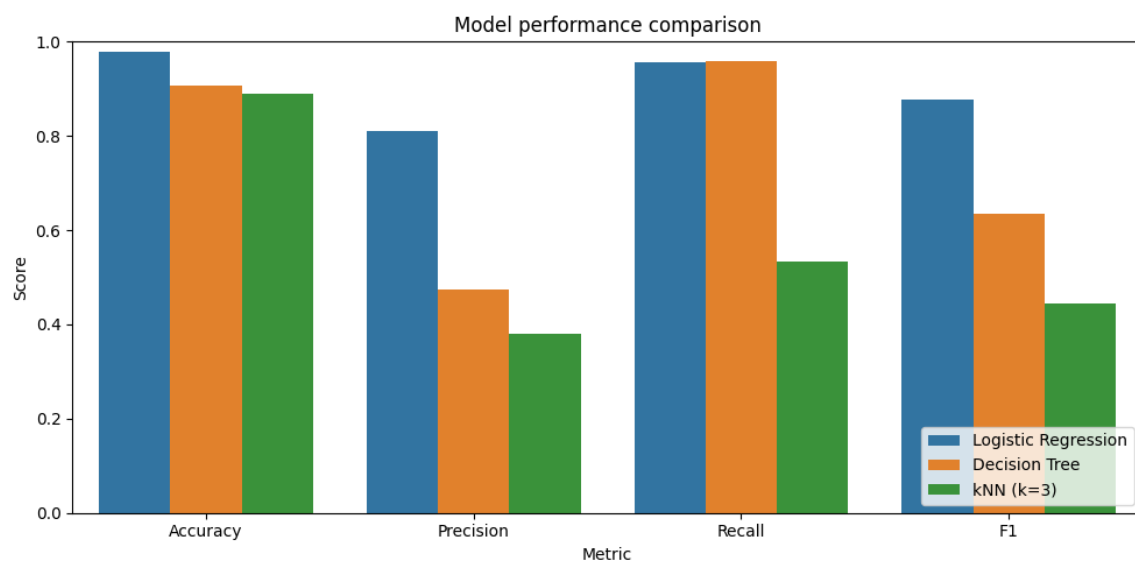


Figure 6. Model performance comparison across key metrics.

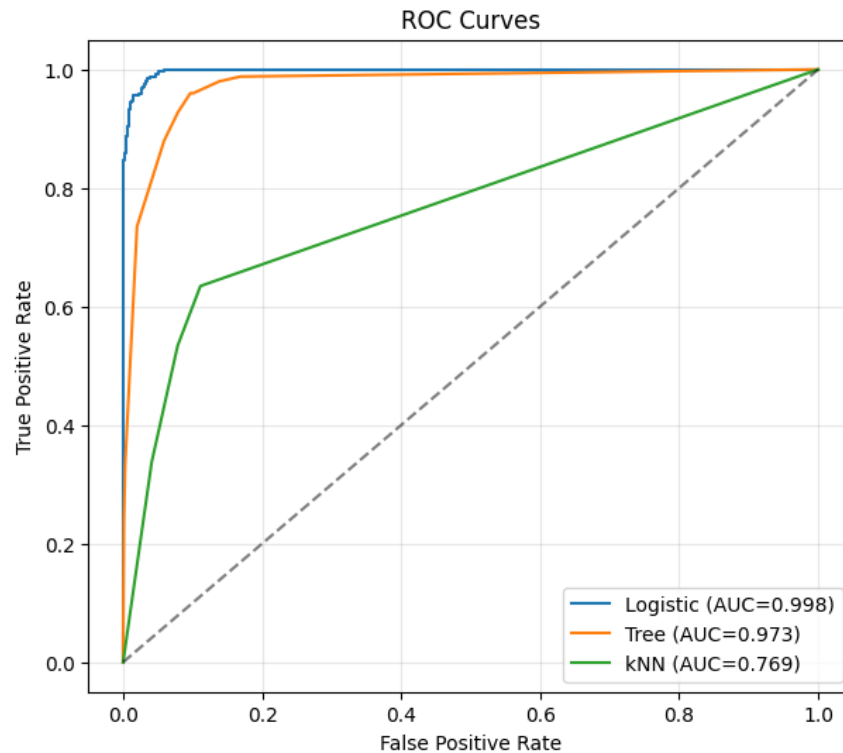


Figure 7. ROC curves for all three models.

Logistic regression beats the other two on every metric that matters for this problem. The decision tree matches logistic on recall but trails badly on precision. kNN is behind on both. Looking at ROC AUC, logistic regression at 0.998 has near-perfect ability to rank claims by fraud risk, which means even if we wanted to change where we draw the cutoff (say, to catch even more fraud at the cost of more false alarms), we'd have a lot of room to do that.

7. What the Model Learned

Beyond just knowing which model performs best, it's worth looking at what features it relies on. For logistic regression, the coefficients tell us this directly.

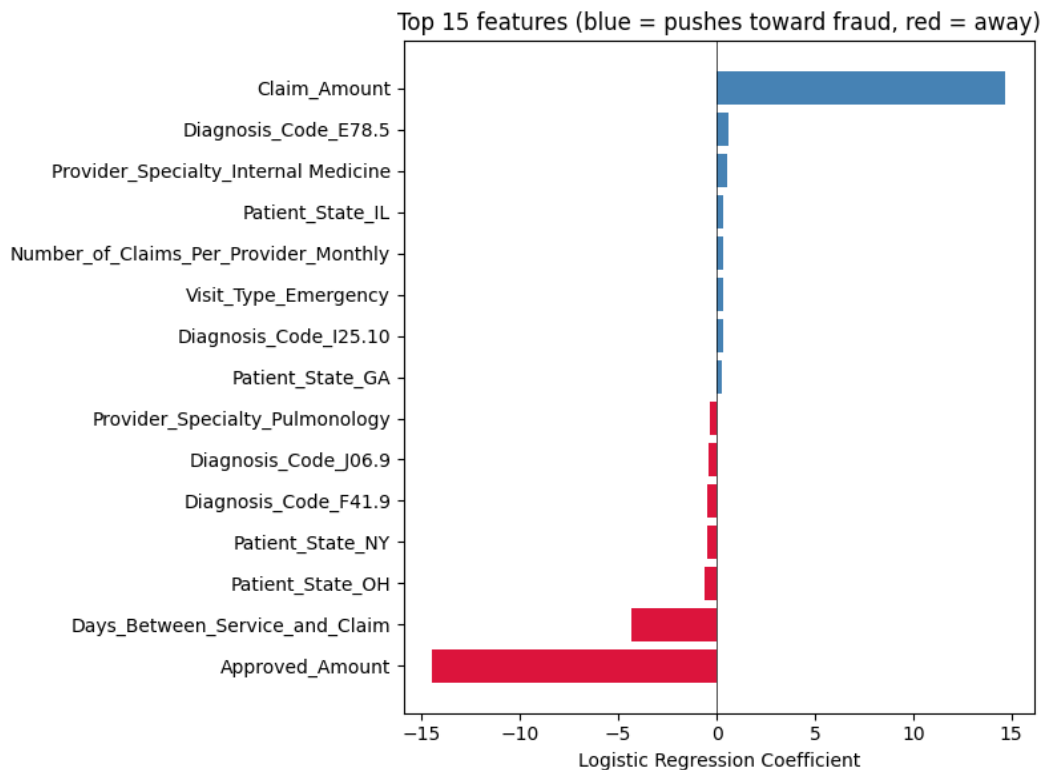


Figure 8. Top 15 features by logistic regression coefficient magnitude.

Two financial features dominated: claim amount (large positive coefficient, meaning higher amounts push toward a fraud prediction) and approved amount (large negative coefficient). Together these are essentially capturing the gap between what gets billed and what gets approved, which is one of the classic signals investigators look for. A claim where the billed amount is much higher than the eventual approved amount is exactly the kind of thing that suggests something is off.

After that, the gap between when the service was provided and when the claim was filed shows up as the next strongest signal, with a negative coefficient, because claims filed quickly are more suspicious than claims filed with a normal lag. This also makes intuitive sense; legitimate claims often go through batch processing and take longer, while suspicious patterns can show up as unusually fast filings.

The remaining top features were combinations of provider specialty, diagnosis code, and patient state. None of these individually were huge drivers, but together they captured the patterns of who tends to be involved when fraud happens. We didn't see any single "smoking gun" feature in this group, which is reassuring, meaning the model is making decisions based on a combination of evidence rather than relying on one shortcut.

8. Benchmark Sanity Check

When we got our interim report feedback, Prof. Bala suggested comparing our performance to what's already out there on this dataset, since Kaggle datasets often have public solutions that set a benchmark. We took a look.

This particular dataset (nudratabbas) is fairly new and doesn't yet have a deep ecosystem of public notebooks to compare against directly. But the broader literature on healthcare fraud detection is consistent on a few things: accuracy alone is misleading because of class imbalance, F1 score and recall are the right metrics to focus on, and typical reported F1 scores on similar imbalanced healthcare fraud datasets sit somewhere between 0.75 and 0.90 when imbalance is handled properly. Our logistic regression F1 of 0.88 lands comfortably inside that range, which suggests we're getting reasonable performance without either overfitting or accidentally introducing leakage. Catching 96% of fraud with 81% precision is a strong result for this kind of problem, and it would be unusual to see materially better numbers without resorting to ensemble methods like gradient boosting.

9. Business Implications

If an insurer were to deploy our logistic regression model as a screening tool, here's what they would actually see in practice.

Out of every 3,000 claims processed, the model would flag roughly 294 for investigator review. Of those, about 238 would be real fraud and 56 would be false alarms. The remaining 2,706 claims would pass through without additional review. Of those, 11 would be fraudulent claims that slipped through undetected. So the trade-off looks like this: investigators spend their time on a manageable workload (about 10% of all claims), they catch the large majority of fraud, and they do it with a reasonable signal-to-noise ratio.

For an insurance company, this kind of system would pay for itself many times over. Even at modest claim amounts, preventing 96 out of every 100 fraud cases from being paid out would translate to substantial savings. And because the model is flagging claims for human review rather than auto-denying them, the system stays inside the kind of human-in-the-loop guardrails that fraud investigation processes typically require for compliance and appeal purposes.

The model also gives investigators something to work with beyond a yes/no flag. Because logistic regression produces a probability, the same model could be used to prioritize a queue, with the most suspicious claims worked first, then the moderately suspicious ones, and so on. That kind of workflow integration is usually what makes the difference between a model that gets used and one that sits on the shelf.

10. Limitations and Future Work

A few honest caveats are worth flagging. For one, this is a synthetic Kaggle dataset, not real claims data, so the patterns the model is picking up on may be cleaner and more learnable than they would be in the wild. Real-world fraud is messier; fraudsters adapt, and patterns shift over time. Any production system would need to be retrained on fresh data on a regular basis.

We also worked with only 10,000 rows. That's enough to build and compare models, but a real insurance company would have orders of magnitude more data, and that would change which methods are practical. For instance, gradient boosting methods like XGBoost typically do very well on this kind of tabular problem at scale, and would be a natural next step if we had more time and more data.

On the imbalance side, we used random oversampling rather than SMOTE. Random oversampling is simpler and worked well for our needs, but SMOTE (which synthesizes new minority examples instead of just duplicating) is generally considered more sophisticated and could be worth comparing in a follow-up. We also didn't do any cost-sensitive analysis where false negatives are explicitly weighted more heavily than false positives, which would be appropriate for a true business deployment.

And on features, we engineered them only minimally. With more time, we'd want to build provider-level aggregations (e.g., that provider's historical fraud rate, average claim size, year-over-year change in monthly claim volume) since the literature suggests those are often the strongest signals in real fraud detection systems.

11. Conclusion

We started this project wanting to see whether straightforward data mining methods could meaningfully identify healthcare fraud, and the answer is yes. Logistic regression, paired with random oversampling to handle the class imbalance, caught nearly all the fraud cases in our test set with a precision rate that would be perfectly workable in an operational setting. The decision tree and kNN models lagged behind, but each gave us useful perspective on the problem. The tree showed us that the patterns aren't simple enough to be captured cleanly by axis-aligned splits, and kNN showed us that the fraud cases aren't simply clustered together in feature space.

More importantly, the model was learning patterns that lined up with how human investigators think about fraud. Unusual gaps between billed and approved amounts, suspiciously fast filings, and certain combinations of provider specialty and patient history all came through as the strongest signals. That alignment between model and intuition is what makes us comfortable saying this isn't just a number-chasing exercise; the model is picking up on the right things.

For an insurer with limited investigator bandwidth, a model like this would be a meaningful productivity multiplier that catches more fraud, wastes fewer hours, and runs a screening process that is defensible and explainable. It's not a finished product. There's plenty more we'd want to do with more time and real data, but as a first pass, the results are encouraging.

Appendix: Dataset and Code

Dataset: Healthcare Fraud Detection Dataset by nudratabbas, available at <https://www.kaggle.com/datasets/nudratabbas/healthcare-fraud-detection-dataset>

Code: The full Jupyter notebook (Healthcare_Fraud_Detection_Final.ipynb) is submitted alongside this report. It contains all data preparation, model fitting, evaluation code, and the plots reproduced above.